

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING COURSE
CODE: CSA4305

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Short Answer Test

Weightage: 10%

QUESTIONS:

Total Marks: 20

Question 1 (5 Marks)

Suppose that a company wants its website to work seamlessly on mobile, tablet, and desktop devices. Explain the concept of responsive web design and its importance. Describe the key CSS techniques used to achieve responsiveness, and explain how CSS Flexbox or Grid can be used to design adaptive layouts. Illustrate your answer with a simple example layout and suggest improvements to enhance user experience across different devices.

Question 2 (5 Marks)

Suppose that a webpage contains a form where user input must be validated before submission. Explain the concept of client-side validation and its purpose. Describe how JavaScript event handling is used in validation and explain the role of regular expressions in validating inputs

such as email or phone numbers. Provide a suitable example and suggest improvements for better usability and security.

Question 3 (5 Marks)

Suppose that a webpage layout breaks when the browser window is resized. Explain the CSS box model and its components, and describe how CSS positioning techniques affect layout structure. Identify possible causes of layout breaking and explain how media queries can be used to resolve these issues. Suggest best practices to ensure a stable and responsive layout.

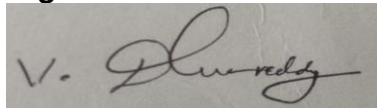
Question 4 (5 Marks)

Suppose that a website requires dynamic content updates without reloading the entire page. Explain the concept of the Document Object Model (DOM) and its role in web development. Describe how JavaScript can be used to manipulate the DOM for updating webpage content dynamically. Provide an example and suggest improvements to enhance performance and user experience.

Code of Conduct:

I v . d h a r a n i r e d d y 1 9 2 3 1 1 2 1 9 _ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A handwritten signature in black ink on a light-colored background. The signature appears to be 'V. Dharamireddy' written in a cursive, flowing style.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with	Incorrect or irrelevant answer

				noticeable errors	
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response

V. Dharmaraj
192311219
CSE

CSA4305- INTERNET PROGRAMMING 4108126

AT-03-C02 - Assignment.

Answers:

1M Responsive web design

Definition:

Responsive web design is a web design approach that makes a website automatically adjust its layout, images and content to fit different screen sizes such as mobile, tablet and desktop.

Importance of Responsive web design:

- Works on all devices
- Improves user experience
- Faster and easier navigation
- Reduces zooming and scrolling
- Improves SEO
- Saves development and maintenance cost.

Key CSS Techniques for responsiveness:

1. Media Queries

- Apply different CSS styles for different screen sizes

Q:

```
@ media (max-width: 768px) {
```

```
  .container {
```

```
    flex-direction: column;
```

```
  }
```

```
}
```

Flexible layouts

- Use percentages (%) instead of fixed pixels.

Ex: width: 80%

3. Flexible Images

→ images resize automatically.

```
img {  
  max-width: 100%;  
  height: auto;  
}
```

Using CSS Flexbox:

Ex: code

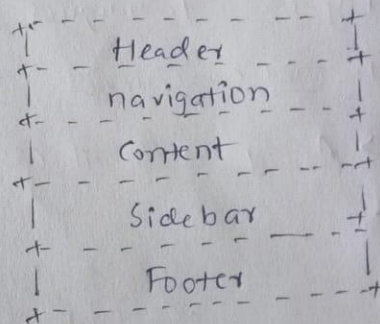
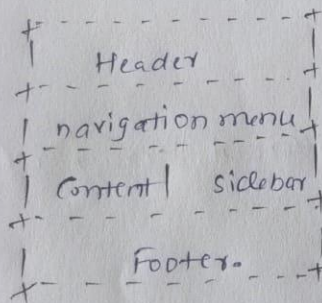
```
• Container {  
  display: flex;  
  gap: 10px;  
  flex-wrap: wrap;  
}  
• box {  
  flex: 1;  
  padding: 20px;  
}
```

Using CSS Grid:

Ex: code

```
• Container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;  
}  
@media (max-width: 768px) {  
  • Container {  
    grid-template-columns: 1fr;  
  }  
}
```

Simple Example layout:



Improvements for better user experience:

- Use responsive navigation
- Increase button and text size on mobile
- Optimize images for faster loading
- Use readable font and colours.

Client - Side Validation:

Definition:

→ Client side validation is the process of checking user input in the browser using javascript before the submission of the form to the server.

Purpose of client side validation;

- Prevents Invalid data entry
- Improves user experience
- Reduces server load
- Provide instance error messages
- Saves time by avoiding unnecessary form submission.

JavaScript event Handling in Validation;

→ Javascript uses event to validate input fields.

Common Events:

- onsubmit → check all fields before submitting
- oninput → validates while typing
- onblur → validates when user leaves a field
- onclick → validates when a button is clicked.

Code:

(HTML)

```
* <form onsubmit="return validateForm()">  
  name: <input type="text" id="name">  
  <input type="submit" value="submit">
```

```

</form>
<script>
function validateForm() {
let name = document.getElementById("name").value;
if (name == "") {
alert("Name is required");
return False; }
return True; }
</script>

```

Role of regular Expressions (regex)

→ regular expressions are patterns used to check whether user input matches a required format.

Email Validation:

```

let
emailPattern = /^[A-Za-z0-9]+[-/+.]+@[A-Za-z0-9]+[.][A,
-Z, a-z]{2,3}$/;

```

Phone number Validation:

```

let phonePattern = /^[0-9]{10}$/;

```

Example Validation:

Input:

→ Name: Dharani
→ Email: abc@gmail.com
→ Phone: 9876543210

If invalid:

→ Email: abcdmail.com
→ Phone: 98765

Output: Form submitted successfully Output: Enter valid email and no.

Improvements for Better usability and security:

- * Show clear error messages
- * Highlight invalid fields
- * validate inputs in real time
- * use placeholders at input hints
- * Disable submit button until all fields are valid.

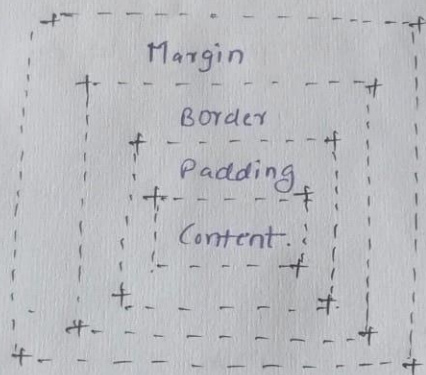
ANSWER:

CSS Box Model and Responsiveness layout:

CSS BOX Model:

The CSS Box model defines how size of an HTML element is calculated. Every element consists of four parts;

- Content - Actual text or image
- Padding - space between content and border
- Border - Surrounds padding and element content.
- Margin - space outside the border.



CSS positioning Techniques:

- Static: Default position
 - Relative: moves relative to its original position
 - Absolute: positioned relative to nearest positioned
 - Fixed: stays fixed on the screen while scrolling.
 - Sticky: Acts as relative until scrolling, then becomes fixed.
- ⇒ This affects how elements are arranged on webpages.

Causes of Layout Breaking:

- Fixed width (width: 500px) instead of %.
- Large Images without resizing
- Incorrect use of absolute positioning
- missing media queries
- Content overflow due to long text
- Using too many fixed margins and padding.

Using media queries:

- media queries apply different CSS styles for different screen sizes.

Example:

Code:

```
• container {  
  display: flex;  
}  
@media (max-width: 768px) {  
  • container {  
    flex-direction: column;  
  }  
}
```

Best Practices:

- Use Flexbox or CSS grid.
- Use percentage (%) or rem units instead of fixed pixels.
- Make images responsive using max-width: 100%.
- Apply media queries like for mobile, tablet and desktop.
- Test the website on different screen sizes.

Answer:

Document Object Model (DOM) and Dynamic Content:

DOM:

Definition:

→ The document object model is a tree-like structure created by the browser that represents all HTML elements of a webpage. Javascript uses the DOM to access, modify and add or to remove elements without reloading pages.

Role of DOM in web development:

- Access HTML elements
- change text, images and styles
- Add or remove elements dynamically
- Handle user interactions.
- Update webpage content without refreshing.

Javascript DOM manipulation:

Javascript uses methods like;

- `getElementById()`
- `querySelector()`
- `innerHTML()`
- `textContent`
- `style.`

Example

Code HTML:

```
<p id="msg">welcome!</p>  
<button onclick="changeText()">click Me</button>
```

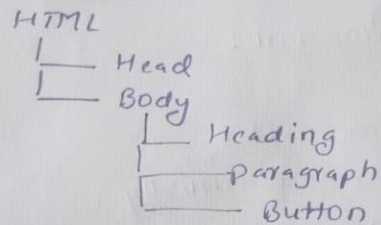
```
<script>
function changeText () {
document.getElementById("msg").innerHTML = "Hello,
user!";
}
</script>
```

Output:

Before clicking : Welcome!

After clicking : Hello, user!

Simple DOM Structure:



→ Javascript access these elements through the DOM and updates them instantly.

Improvements for better performance and user experience:

- Update only required elements instead of reloading page.
- minimize unnecessary DOM operations.
- use event listeners instead of inline events
- optimize images and scripts for faster loading
- show loading indicators when fetching data.
- use smooth animations and clear user feedback.